

Strategic Execution Plan: AI-Driven Anomaly Detection in Component Burn-In & Screening (SIH 26170)

The Indian Space Research Organisation (ISRO) requires a smart automation tool to detect anomalies in aerospace electronic components during thermal and electrical burn-in testing. Traditional static thresholds fail to catch subtle degradation patterns that lead to component failure in space.

This document outlines the rapid-prototyping execution plan to build a functioning, AI-driven anomaly detection dashboard from scratch in exactly **6 days**. To ensure a high probability of success for a beginner-to-intermediate team, the architecture explicitly avoids complex microservices and relies entirely on a vanilla technology stack (HTML, CSS, JS, Python, SQL).

Member 1: Hardware Simulation & Data Engineer

What Member 1 Does (In Plain English) Think of Member 1 as the person building the "virtual test chamber and the physical components." Real burn-in testing takes weeks and requires physical microchips and ovens. Member 1 writes Python code that mathematically simulates a microchip getting hot and drawing electrical current over time. If a teammate needs to see what a "short circuit" or a "slowly melting wire" looks like in data form, Member 1 writes the code that generates those fake numbers.

Tech Stack & Tools

- **Primary Language:** Python 3.x
- **Libraries:** random, math, csv, sqlite3
- **Data Output:** CSV files (for AI testing) and SQL inserts.

5-Day Detailed Work Plan

Day	Focus Area	What to Build & How to Do It
Day 1	Component Baseline Logic	* Define the basic component variables: Voltage (V), Current (I), and Temperature (T). * Write a loop that generates normal, safe operating numbers (e.g., $T = 85^{\circ}\text{C} \pm 0.5$).
Day 2	Sensor Noise & Fluctuation	* Add mathematical "jitter." Real sensors are not perfectly stable. Inject tiny, random fluctuations into the numbers using random.uniform() so the data looks real.
Day 3	Latent Defect Injection (Drift)	* Write a toggleable function for "Thermal Runaway." If activated, the temperature

Day	Focus Area	What to Build & How to Do It
		baseline slowly increases by 0.05°C every second, simulating a failing heatsink.
Day 4	Catastrophic Failure Injection	* Write a toggleable function for an "Electrical Short." If activated, the Voltage drops instantly while the Current spikes massively.
Day 5	Static CSV & SQLite Generation	* Run the script to generate 10,000 rows of normal data, and 1,000 rows of failing data. Save this as sample_data.csv and give it to Members 2 and 3 so they can test their math without needing the live server.

Key Deliverable by End of Day 5: A Python script (simulator.py) that can simulate a virtual burn-in chamber. It can output clean, realistic telemetry and purposefully inject both slow-drift errors and sudden-spike errors on command.

Member 2: Multivariate ML Engineer

What Member 2 Does (In Plain English) Think of Member 2 as building the "sudden spike and chaos detector." Sometimes, a component fails by doing something entirely unpredictable—like the temperature dropping while the current spikes. Member 2 uses a machine learning algorithm called an Isolation Forest to instantly recognize when the combination of numbers is acting weird, even if no single number has broken a hard limit.

Tech Stack & Tools

- **Primary Language:** Python 3.x
- **Libraries:** scikit-learn (IsolationForest)
- **Core Rule:** Do not touch the APIs. Just read Member 1's CSV.

5-Day Detailed Work Plan

Day	Focus Area	What to Build & How to Do It
Day 1	Algorithm Setup & Research	* Import <code>sklearn.ensemble.IsolationForest</code> . * Read the documentation on how it isolates anomalies through recursive splitting.
Day 2	Model Training on CSV	* Write a script that loads Member 1's sample_data.csv. * Train the Isolation Forest model strictly on the "normal" data so it learns what a healthy component looks like.
Day 3	The Detection Function	* Write a clean, isolated function: <code>def detect_spike(current, voltage,</code>

Day	Focus Area	What to Build & How to Do It
		temp):. * Have the function pass those three numbers into the trained model.
Day 4	Output Standardization	* If the model returns -1, it means an anomaly was found. Have your function return a clean True or False.
Day 5	Tuning & False Alarms	* Test the function against Member 1's "Catastrophic Failure" data. Tweak the contamination parameter until it catches the short circuits immediately without falsely flagging normal noise.

Key Deliverable by End of Day 5: A single Python file (isolation_forest.py) containing a function that takes an array of sensor numbers and returns True if it detects a chaotic electrical anomaly.

Member 3: Time-Series AI Specialist

What Member 3 Does (In Plain English) Think of Member 3 as building the "slow creep and degradation detector." If a component's safe temperature limit is 90°C , and it is currently at 85°C , standard alarms do nothing. But if it is slowly creeping up by 0.1°C every minute, it *will* fail. Member 3 writes the math (CUSUM) that notices this slow, tiny creep and sounds the alarm *before* the component hits 90°C .

Tech Stack & Tools

- **Primary Language:** Python 3.x
- **Libraries:** Standard Python Math
- **Core Rule:** Do not touch the APIs. Just read Member 1's CSV.

5-Day Detailed Work Plan

Day	Focus Area	What to Build & How to Do It
Day 1	CUSUM Theory & Setup	* Research the Cumulative Sum (CUSUM) algorithm. You need variables for the target mean (μ), standard deviation (σ), and sensitivity (k).
Day 2	Stateful Tracking Logic	* Write a Python class DriftDetector. It must remember the previous calculations to maintain a "cumulative sum" of the errors over time.
Day 3	The Detection Function	* Write the core math: $S^+_n = \max(0, S^+_{n-1} + X_n - (\mu + k))$. * If the cumulative drift (S^+) breaks a threshold limit, trigger the alarm.

Day	Focus Area	What to Build & How to Do It
Day 4	Output Standardization	* Add a function <code>def evaluate_drift(sensor_value):</code> that takes one number, updates the internal memory, and returns True if drift is critical.
Day 5	Tuning & Calibration	* Run the function against Member 1's "Latent Defect" CSV data. Prove that your math catches the slow temperature rise minutes before it actually hits the hard failure limit.

Key Deliverable by End of Day 5: A single Python file (`cusum_drift.py`) containing a stateful class that remembers past numbers, tracks slow degradation, and flags anomalies before thresholds are breached.

Member 4: Backend API Developer

What Member 4 Does (In Plain English) Think of Member 4 as the "post office and the data bridge." Python can crunch the numbers, but web browsers cannot run Python. Member 4 builds a lightweight server that takes the Python telemetry, packages it into a JSON envelope, and pushes it through a real-time pipe (a WebSocket) to the web browser so the UI can display it.

Tech Stack & Tools

- **Primary Language:** Python 3.x
- **Libraries:** FastAPI, uvicorn, json
- **Core Rule:** Do not integrate the AI models yourself. Just build the pipes using dummy text.

5-Day Detailed Work Plan

Day	Focus Area	What to Build & How to Do It
Day 1	Server Foundation	* Install FastAPI and Uvicorn. Write <code>main.py</code> and set up the basic app instance.
Day 2	Database Query Endpoints	* Write standard HTTP endpoints (e.g., <code>@app.get("/history")</code>) that can execute basic SQLite SELECT queries to retrieve old data.
Day 3	WebSocket Integration	* Write the real-time endpoint: <code>@app.websocket("/ws")</code> <code>[span_8](start_span)[span_8](end_span).</code> * Inside, create an <code>async def</code> function that accepts incoming client connections.
Day 4	Dummy Data Broadcasting	* Write a <code>while True:</code> loop inside the WebSocket endpoint. Every 1 second, use <code>await websocket.send_text()</code> to push

Day	Focus Area	What to Build & How to Do It
		a hardcoded JSON string like <code>{"temp": 85, "volt": 5.0, "status": "safe"}</code> .
Day 5	Connection Management	* Add a try/except block to catch WebSocketDisconnect errors so the server does not crash if the user closes their browser window.

Key Deliverable by End of Day 5: A working FastAPI server that boots up on localhost:8000 and continuously broadcasts dummy JSON data over a WebSocket connection without crashing.

Member 5: UI/UX Web Designer

What Member 5 Does (In Plain English) Think of Member 5 as the person building the "mission control monitors." Member 5 builds the aesthetic, structural layout of the dashboard. They do not write the logic that makes the charts move; they build the sleek, dark-mode containers that hold the charts, and the flashing alert boxes that will eventually display the AI warnings.

Tech Stack & Tools

- **Languages:** HTML5, CSS3
- **Core Rule:** Write absolutely no JavaScript logic. Focus entirely on the visual layout.

5-Day Detailed Work Plan

Day	Focus Area	What to Build & How to Do It
Day 1	UI Skeleton & Grid	* Build index.html. Use CSS Grid or Flexbox to split the screen: Header on top, big panel in the center (for charts), side panel on the right (for alerts).
Day 2	Theming & Dark Mode	* Write style.css. Apply an aerospace aesthetic: deep charcoal backgrounds, neon green text for safe statuses, and monospaced fonts (like Courier or Roboto Mono).
Day 3	Canvas & HTML Setup	* Inside the center panel, place a large <code><canvas id="telemetryChart"></canvas></code> tag. This provides the blank slate for Member 6 to draw the live graphs later.
Day 4	Alert Feed Design	* Build the right-side panel as an "AI FDIR Log." Give it an ID <code><div id="alert-feed"></div></code> . * Create CSS classes for

Day	Focus Area	What to Build & How to Do It
		.alert-red and .alert-yellow that Member 6 can dynamically apply when anomalies happen.
Day 5	Scenario Control Panel	* Build a small control panel in HTML with buttons like <button id="btn-thermal">Inject Thermal Defect</button>. (Do not wire them up, just make them look good).

Key Deliverable by End of Day 5: A beautiful, static HTML/CSS webpage that looks exactly like an ISRO quality-assurance dashboard, ready for data injection.

Member 6: Database & Frontend Integrator (Lead)

What Member 6 Does (In Plain English) Think of Member 6 as the "chief conductor of the orchestra." Members 1-5 built specialized, isolated pieces. None of them work together yet. Member 6 is the only person who writes the JavaScript to animate Member 5's website, and on Day 6, they stitch everyone's Python code together to form the final, working product.

Tech Stack & Tools

- **Languages:** JavaScript (Vanilla), HTML, Python (Integration)
- **Libraries:** Chart.js (via CDN)
- **Core Rule:** Manage the data flow and assemble the prototype.

5-Day Detailed Work Plan (Pre-Integration)

Day	Focus Area	What to Build & How to Do It
Day 1	Database Architecture	* Finalize the burn_in.db SQLite schema. Ensure the telemetry table has columns for timestamp, current, voltage, temperature, and anomaly_score.
Day 2	Chart.js Setup	* Write app.js. Include Chart.js via CDN in Member 5's HTML. * Initialize a blank line chart inside the <canvas> tag, tracking Temperature and Voltage.
Day 3	Dummy Array Animation	* Write a JS setInterval loop that pushes fake numbers into the Chart.js dataset and calls chart.update(), making the graph scroll left-to-right smoothly.
Day 4	DOM Alert Manipulation	* Write JS functions that can manipulate the DOM. If a test variable is True, use document.getElementById('aler

Day	Focus Area	What to Build & How to Do It
		t-feed').innerHTML to inject a red warning into Member 5's UI.
Day 5	Demo Scripting & Review	* Write the presentation script. Prepare the narrative explaining the difference between standard thresholds and AI-driven predictive maintenance.

Day 6: Master Integration & Demo Preparation On Day 6, Member 6 brings all the code together into the single continuous loop:

[Member 1: Hardware Simulator] $\rightarrow$ [Member 2 & 3: AI Math Scripts] $\downarrow$
 [Member 4: FastAPI Server / WebSocket] $\rightarrow$ [Member 6: JS Web Client] $\downarrow$
 [Member 5: HTML/CSS Dashboard] $\leftarrow$ [Member 6: Chart.js Visuals]

- **Connect the Backend:** Member 6 opens main.py (Member 4) and imports simulator.py (Member 1), isolation_forest.py (Member 2), and cusum_drift.py (Member 3).
- **Rewrite the Loop:** Change the WebSocket loop from sending "dummy data" to pulling the real generated numbers, passing them through the AI functions, and broadcasting the real anomaly scores.
- **Connect the Frontend:** In app.js, Member 6 writes `const ws = new WebSocket("ws://localhost:8000/ws")` to catch the live data and feed it directly into the scrolling Chart.js graphs.

The Final Prototype Demo Experience

For the hackathon presentation, the judges will see a live, browser-based Mission Control Center built entirely with HTML, CSS, and JS.

The Hook (How to pitch it): "In high-stakes space missions, waiting for a component's temperature to break a red-line limit is too late—the hardware is already damaged. Our system doesn't wait for failures; it uses AI to detect the microscopic symptoms of degradation before the threshold is ever breached."

The Interactive Demo Scenarios:

Scenario	What the Prototype Will Do
Nominal Burn-In	The HTML dashboard displays scrolling Chart.js graphs of current, voltage, and temperature hovering safely in the green. The FDIR log is clear.
Latent Thermal Defect	The presenter clicks "Inject Thermal Drift." The temperature graph slowly, almost imperceptibly, angles upward. Within seconds, Member 3's CUSUM AI detects the unnatural trend, flashing a yellow warning on the UI <i>minutes</i> before the temperature hits the 90°C hard-limit.
Sudden Micro-Short	The presenter clicks "Inject Spike." The voltage chart flickers momentarily. Member 2's Isolation Forest AI immediately isolates the

Scenario	What the Prototype Will Do
	chaotic multivariate combination, flashing a red CRITICAL FAILURE alert on the UI and halting the simulated burn-in.

This setup guarantees an organized, conflict-free 6-day sprint, culminating in a highly visual, AI-driven software prototype that directly solves the SIH 26170 problem statement.

Works cited

1. IsolationForest example — scikit-learn 1.9.0 documentation, https://scikit-learn.org/stable/auto_examples/ensemble/plot_isolation_forest.html 2. WebSockets on Render, <https://render.com/docs/websocket>